

Nathaniel Abera

August 19, 2026

Software Security and Testing

A1. Static code analysis is an inspection that is made of the source code without the need of executing the program it will be able to find any flaws including vulnerabilities or other defects. It gives a deep analysis of any issues that need to be attended before a code is launched.

This is mainly used within secure software development as it is done as an early checkpoint because it gives developers an understanding of anything important that needs to be adjusted within the source code before it is exposed by operating users. It gives a consistent observation of the entire codebase and gives direct detail on things that need to be reviewed or tested.

A2. The first issue that static analysis is supposed to detect is violations. These are errors found within the language formatting, such as missing blank lines between functions or lines that exceed the maximum length allowed (Cordasco, 2016). These don't really break the code but it does make it harder to read and will make it harder for someone to fix anything within it.

The second issue that static analysis is supposed to detect is the unused code which includes things such as unreferenced variables or imports that aren't being used. Within Flake8, it had flagged unused imports and variables within the application and these are often indications of incomplete code since it was never used in the first place, which means that it was either not setup properly or never finished.

The third issue static analysis should be detecting is security vulnerabilities. This involves detecting any patterns that expose the application in ways that could compromise user credentials or leak the source code of the application itself. There were two flags that bandit was able to discover and they were both hardcoded values that were made within the application. If secrets were stored like this then it would easily be accessible to anyone and they would gain access to the source code which would compromise the application entirely and expose the credentials of its users.

The fourth issue that static analysis needs to detect is error constructs which are statements that behave oddly under specific conditions. The usage of assertions causes bandit to flag it because it removes when Python runs in “optimized mode”, which means a check that appears to protect the code may not exactly execute at all in production (Bandit Developers, 2026).

The fifth and last issue that static analysis should be detecting is error paths that are being unhandled, as these are operations that can raise an exception with no handling in place, such as dictionary lookups or a type conversion that’s performed on a user input. This would result in a termination of a request rather than giving a response that is controlled.

A3. The first issue that I had identified is in relation to the storage of credentials. They are being stored as plain-text literals in the source code, including the Flask secret key and the password “secret123”. Both values are written directly into the application file, so anyone able to view the source code could authenticate as a valid user or forge session cookies.

The second issue that I had identified was the missing exception handling for user supplied input. The rental route looks up the equipment type in the pricing dictionary and converts the durations that have been submitted into an integer containing no protection, so any unknown equipment names will raise a KeyError and a duration that doesn’t have a number will raise a ValueError. Either of these conditions will crash the request, which allows invalid inputs to disrupt availability.

The third issue that I had also found was a flaw within the logic of the cost calculator. The daily rate is multiplied by the total number of days that were submitted, but the application itself does not confirm whether or not the value is positive. If a user submitted a negative duration, the application would calculate a negative total and will still confirm the rental as successful, which allows a user to generate a credit against the system.

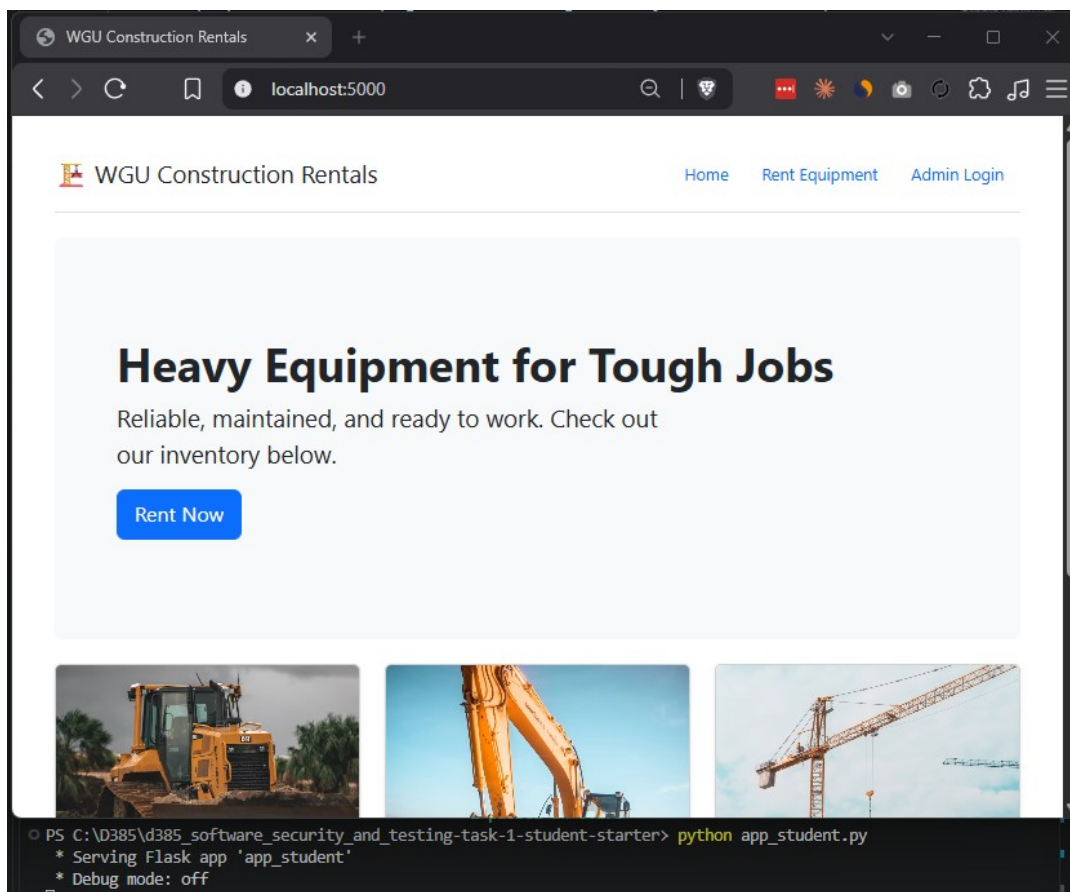
The fourth issue which I identified is that the requested IP is captured by the login route but never evaluates the IP address. This means that the application cannot distinguish whether an IP

address is coming from an internal or external traffic source and it does not record where an authentication attempt originates. This removes the main signal being used to detect brute force attempts.

The fifth issue that I identified is that the application itself uses print statements instead of using the logging framework. This causes the console output to not be written on the disk and causes it to be lost when an application restarts. Because of this, failed logins and application errors will leave no audit trail for future reviews.

B4.

Figure 1- Application running on localhost:5000



B5.

Figure 2: Expected exception handling – Invalid equipment type test passed

```
PS C:\D385\d385_software_security_and_testing-task-1-student-starter> pytest test_run.py::test_invalid_equipment_type_handled -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\hanak\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\D385\d385_software_security_and_testing-task-1-student-starter
collected 1 item

test_run.py::test_invalid_equipment_type_handled PASSED [100%]

===== 1 passed in 0.29s =====
```

Figure 3: Expected exception handling – Non-numeric duration test passed

```
PS C:\D385\d385_software_security_and_testing-task-1-student-starter> pytest test_run.py::test_non_numeric_days_handled -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\hanak\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\D385\d385_software_security_and_testing-task-1-student-starter
collected 1 item

test_run.py::test_non_numeric_days_handled PASSED [100%]

===== 1 passed in 0.33s =====
```

Figure 4: Input validation – Negative duration rejected

```
PS C:\D385\d385_software_security_and_testing-task-1-student-starter> pytest test_run.py::test_negative_days_rejected -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\hanak\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\D385\d385_software_security_and_testing-task-1-student-starter
collected 1 item

test_run.py::test_negative_days_rejected PASSED [100%]

===== 1 passed in 0.29s =====
```

Figure 5: Correct function output – Rental total calculated correctly

```
PS C:\D385 Task 1 Files - Nathaniel Abera\d385_software_security_and_testing-task-1-student-starter> pytest test_run.py
::test_student_correct_output -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\hanak\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\D385 Task 1 Files - Nathaniel Abera\d385_software_security_and_testing-task-1-student-starter
collected 1 item

test_run.py::test_student_correct_output PASSED [100%]

===== 1 passed in 0.28s =====
```

C1a. The log entries revealed problems that the original application could not surface at all, because it wrote significant events to the console instead of doing it to a file. Once logging was setup every request produced a record of a timestamp which showed an input of what was submitted and its result. Rental durations that showed an error within the log were identified with the value as rejected and a failed authentication appears as a warning with additional entries showing the source address and the account. By reviewing these entries in sequence, it is now possible to view what inputs are causing failures, how often they happen, and which ones are the same ones occurring repeatedly that need to be diagnosed and fixed accordingly.

C1b. The first pattern that appears to be abnormal was the six failed login attempts that were recorded between 18:55:06 and 18:55:40. These attempts were targeting three different account user names that were admin, administrator, and root. All the attempts produced a warning entry. This discovery matches a pattern of suspicious activity since a legit user wouldn't attempt three different account names in one succession. It also shows that the login doesn't have any rate limiting that would lock users out after multiple attempts, so an attacker could use brute force tactics to constantly guess different sequences and gain access once they succeed.

The second pattern that was abnormal was a rapid sequence of rental submissions being attempted at 18:58:15 within a short timeframe, 15ms. The logs show that three separate entries occurred: an unrecognized equipment type that submitted a warning in the logs, a non-numeric duration that was also logged as a ValueError, and a negative duration that was logged as an AssertionError. This discovery indicates that an automated submission was made rather than a manual entry and the variety of different values being attempted gives the impression that

probing occurred for both input fields which doesn't appear to be from an ordinary user error.

C2a. The first technique that I used and would say was the primary one was logging. By adding timestamp entries within the Warning, Info, and Error levels it allowed me to view what requests were received by each input and where it failed. So if a rental was rejected, I would be able to trace exactly where the value was submitted without having to manually reproduce the request.

A step by step execution that I used was the second technique because it allowed me to test the suite manually. I was able to run individual test rather than a group, which allowed me to isolate each variable and see what fix actually worked on its own before I ran them together.

The last technique I had used was leveraging the breakpoints within VSCode because it let me pause any executions inside of the rental route and it let me view the values of any submitted durations and calculate the totals before it even returned any responses.

C2b. Within Figure 2 the defect that shows is an unguarded dictionary lookup within the line `"daily_rate = EQUIPMENT_PRICES[equipment_type]"`. This raised a `KeyError` which had resulted in a server error whenever a user submitted an equipment type that wasn't within the pricing dictionary. I wrapped the lookup in a try and except block which catches the `KeyError` and logs it in as a `WARNING` that identifies it as a rejected value, and it also shows a message to the user saying that the selected equipment type needs to be valid.

In Figure 3, it shows that the defect was an unvalidated conversion type. The line `"days = int(days_str)"` raised a `ValueError` whenever a user had submitted a duration that wasn't numeric. So what I did was add an except `ValueError` branch that would log the invalid value at the Error level instead and it would also display a message saying that the duration must be a whole number. This way the application stays available even after an invalid input has been submitted.

The Figure 4 also has a defect that was a missing check on the sign of the rental duration. The cost calculation multiplied the daily rate by the submitted days without even confirming whether the value was positive. So this caused the negative durations to produce a negative total even

when the application was confirmed as successful. What I did was add an assertion to the line that was “assert days > 0” right before the calculation and I also put an except AssertionError branch that would reject inputs at the Error level and log it.

The figure 5 was the last defect that I saw and it showed that there was an absence of any successful transactions being verified which the original code used to take care of by printing a statement to the display which contained the calculated cost and left no automated confirmations or audit trail. So I replaced the print statement by adding a Info log entry that records the equipment type, duration, and total, and I added a unit test that would act as an assertion rule and allow 3-day bulldozer rental to return the value of \$1500. This would ensure that the values come back correctly without saying something is successful without verifying it first.

C3. The input validation tends to restrict whatever the application accepts before it is even being processed. The reason for this is that the login route will check the submitted username and it will verify it against an allowlist of accounts that have been approved then it would compare the IP address against another list of internal IP address. A warning would appear if an attempt was made from an IP address outside of the internal list. This way it will prevent unauthorized users from misusing the system by blocking accounts that haven’t been approved.

An exception handling rule will keep the application running even when an invalid input makes its way to the calculations. The rental route will now be able to wrap the dictionary lookup as well as the correct type conversion by using a try and except structure with the three core branches we came up with: ValueError for non numeric duration, KeyError for unrecognized equipment, and AssertionError for duration that fails validation. These branches will log any failures and it will give a message to the user about what happened. This will help prevent errors from happening and it will also keep a log of any attempts that were made allowing us to keep a record.

The assertion statements will also enforce any assumptions that the code inherently depends on. The function that validates inputs will also work well with the rental route by asserting the duration is greater than zero before it calculates the cost, as for the username it will be asserted in

a non-empty string. These assertions will pass and write an Info entry within the log. As for the second one it will prevent any act of misuse directly because users now can't submit a negative input and still generate credit.

C4.

The first thing that static code analysis can examine is the source code and that can be done without even running it. This allows for problems to be found before it is ever launched. It can detect any flaws within the code such as security vulnerabilities by putting the code against known insecure patterns that may lead to attackers being able to compromise certain things that have a weak security.

It can also detect any issues within the code such as the functionality of said code. It can detect any incorrect usage of variables that may have never been referenced or operations performed in external inputs that have no protection setup, and imports that were never called. This allows you to find any code that was left incomplete or flawed.

It also allows for style violations to be discovered as well as any errors within the language's formatting will be picked up when it is compared to universal standards. This includes inconsistent spacing or length limits around functions that need to be written in a specific way. As this does not affect the program's functionality it does reduce the chance of having to change it in the future when maintenance is being done within the code itself.

References

Bandit Developers. (2026). B101: assert_used. Bandit documentation.

https://bandit.readthedocs.io/en/latest/plugins/b101_assert_used.html

Cordasco, I. S. (2016). Flake8: Your tool for style guide enforcement.

<https://flake8.pycqa.org/en/latest/>